

运算符重载

1.加号运算符重载

```
#include<iostream>
using namespace std;

class person{
public:
    //成员函数重载加号运算符
    person operator+(person &p)//operator为编译器通用名
    {
        person temp;
        temp.m_a = this->m_a + p.m_a;
        temp.m_b = this->m_b + p.m_b;
        return temp;
    }

    person operator+(int num)
    {
        person temp;
        temp.m_a = this->m_a + num;
        temp.m_b = this->m_b + num;
        return temp;
    }

    int m_a;
    int m_b;
};

//全局函数重载加号运算符
//person operator+(person &p1,person &p2)
//{
//    person temp;
//    temp.m_a = p1.m_a + p2.m_a;
//    temp.m_b = p1.m_b + p2.m_b;
//    return temp;
//}

void test1()
{
    person p1;
    person p2;
    p1.m_a=10;
    p2.m_a=20;
    p1.m_b=20;
    p2.m_b=10;

    //化简形式
    person p3=p1+p2;
```

```

    person p4=p1+50;//运算符重载也可以发生函数重载

    //成员函数重载本质: person p3=p1.operator+(p2);
    //全局函数重载本质: person p3=operator+(p1,p2);

    cout<<p3.m_a<<" "<<p3.m_b<<endl;
    cout<<p4.m_a<<" "<<p4.m_b<<endl;
}

int main()
{
    test1();

    return 0;
}

```

2.左移运算符重载

```

#include<iostream>
using namespace std;

class person{
public:

    //不会用成员函数重载左移运算符，因为无法实现cout在左侧，所以只能用全局函数重载左移运算符
    // void operator<<(cout)//p.operator(cout);简化版本p<<cout;
    // {
    //
    // }

    int m_a;
    int m_b;
};

ostream& operator<<(ostream &cout,person &p)
{
    //如果返回值是 ostream 而不是 ostream&，则会返回一个临时副本，导致链式调用无法正常工作
    //ostream的拷贝构造函数通常是禁用的
    cout<<p.m_a<<" "<<p.m_b<<endl;
    return cout;
}

void test1()
{
    person p;
    p.m_a=10;
    p.m_b=10;

    cout<<p<<"hello world"<<endl;//10 10
}

int main()

```

```

{
    test1();

    return 0;
}

```

3.递增运算符重载

```

#include<iostream>
using namespace std;

class myinteger{

    friend ostream& operator<<(ostream &out,myinteger myint);

public:
    myinteger()
    {
        m_num=0;
    }

    //重载前置++运算符
    //前置递增返回 引用
    myinteger& operator++()//返回引用是为了一直对一个数据进行递增
    {
        ++m_num;
        return *this;
    }

    //重载后置++运算符
    //后置递增返回 值
    myinteger operator++(int)//int代表占位参数，可以用于区分前置和后置递增
    {
        myinteger temp=*this;
        m_num++;
        return temp;
    }

private:
    int m_num;

};

ostream& operator<<(ostream &out,myinteger myint)//ostream为out的数据类型
{
    cout<<myint.m_num<<endl;
    return out;
}

void test1()
{
    myinteger myint;
    cout<<++(++myint)<<endl;
}

```

```

        cout<<myint<<endl;
    }

    void test2()
    {
        myinteger myint;
        cout<<myint++<<endl;//0
        cout<<myint<<endl;//1
    }
    int main()
    {
        test1();

        test2();

        return 0;
    }

```

4.赋值运算符重载

```

#include<iostream>
using namespace std;

class person{
public:
    person(int age)
    {
        m_age=new int(age);
    }

    ~person()
    {
        if(m_age!=NULL)
        {
            delete m_age;
            m_age=NULL;
        }
    }

    person& operator=(person &p)
    {
        //先判断堆区是否有属性，如果有先释放，再进行深拷贝
        if(m_age!=NULL)
        {
            delete m_age;
            m_age=NULL;
        }
        //深拷贝
        m_age=new int(*p.m_age);
        return *this;
    }

    int *m_age;

```

```
};

void test1()
{
    person p1(18);
    person p2(22);
    person p3(20);

    p2=p1; //赋值操作

    cout<<*p1.m_age<<endl;
    cout<<*p2.m_age<<endl;

    p3=p2=p1;

    cout<<*p3.m_age<<endl;
}

int main()
{
    test1();

    return 0;
}
```

5.关系运算符重载

```
#include<iostream>
#include<string>
using namespace std;

class person{
public:
    person(string name,int age)
    {
        m_name=name;
        m_age=age;
    }

    //重载关系运算符==
    bool operator==(person &p)
    {
        if(this->m_name==p.m_name&&this->m_age==p.m_age)
        {
            return true;
        }
        else
        {
            return false;
        }
    }
}
```

```

bool operator!=(person &p)
{
    if(this->m_name!=p.m_name||this->m_age!=p.m_age)
    {
        return true;
    }
    else
    {
        return false;
    }
}
string m_name;
int m_age;
};

void test1()
{
    person p1("zq",22);
    person p2("zq",22);

    if(p1==p2)
    {
        cout<<"p1=p2"<<endl;
    }
    else
    {
        cout<<"p1!=p2"<<endl;
    }
}

int main()
{
    test1();
    return 0;
}

```

6.函数调用运算符重载

```

#include<iostream>
#include<string>
using namespace std;

class myprint{
public:
    void operator()(string test)//重载()
    {
        cout<<test<<endl;
    }
};

class myadd{

```

```

public:
    int operator()(int num1, int num2)
    {
        return num1 + num2;
    }
};

void myprint2(string test)
{
    cout<<test<<endl;
}

void test1()
{
    myprint m_myprint;

    //仿函数
    m_myprint("hello world");//相当于m_myprint.operator()(hello world);

    myprint2("timi");
}

void test2()
{
    myadd m_myadd;

    int ret=m_myadd(100,100);

    cout<<ret<<endl;

    //匿名函数对象
    cout<<myadd()(100,100)<<endl;//类名(): 匿名函数对象，后面进行相应的重载运算符
}

int main()
{
    test1();
    test2();

    return 0;
}

```